$ git help aureliano

Guia de Bolso Git & GitHub

Do primeiro `git init` ao currículo publicado no GitHub Pages — os comandos que aprendi, explicados do meu jeito.

Aureliano Guimarães Belarmino Filho
Ciência de Dados — UNIFEI
Trainee — Asimov Jr · Diretoria de Projetos · Ciência de Dados

Currículo online: https://aurelianogbf.github.io

Capacitação Semana 2 · Novembro de 2026

Sumário

Parte	O que você encontra
1. Entendendo o Git	As três áreas, conceitos e vocabulário básico.
2. Primeiros passos	Instalação, identidade, `git init` e `.gitignore`.
3. O ciclo do commit	`status`, `add`, `commit`, `log` e `diff`.
4. Voltando no tempo	Desfazer mudanças, corrigir commits, `reset` × `revert`.
5. Branches e Git Flow	Ramificações, merge, rebase, conflitos e o modelo da Asimov.
6. Trabalhando com o GitHub	Remotos, push, pull, fetch, clone, colaboradores e Pull Request.
7. Extras	Tags, stash, repositório bare e GitHub Copilot.
8. Como publiquei meu currículo	O passo a passo real usado neste projeto (GitHub Pages).
9. Cola rápida	Tabela-resumo e erros comuns.

> **Convenção:** tudo que estiver entre `<sinais>` deve ser trocado pelo valor real (nome do arquivo, hash do commit, URL...). Linhas iniciadas com `#` nos blocos de código são comentários.

1. Entendendo o Git

Git é um sistema de controle de versão: ele tira "fotografias" (commits) do projeto e guarda todo o histórico na pasta oculta `.git`. **GitHub** é um serviço na nuvem que hospeda repositórios Git e adiciona colaboração: Pull Requests, issues, GitHub Pages, Copilot etc.

O segredo para entender os comandos é saber que existem **três áreas** no seu computador e uma na nuvem:

```
Diretório de trabalho ──git add──▶ Staging (índice) ──git commit──▶ Repositório local ──git push──▶ GitHub
                                                                  GitHub ──git fetch / git pull──▶ Repositório local
```

Termo	Em uma frase
Repositório	Pasta do projeto + seu histórico de versões.
Commit	Um "ponto de salvamento" com autor, data, mensagem e um identificador (hash).
Staging area	Sala de espera: o que foi preparado com `git add` para o próximo commit.
Branch	Linha de desenvolvimento paralela; a principal costuma ser `main` (ou `master`).
HEAD	Ponteiro que indica "onde você está" no histórico.
Remoto / origin	Cópia do repositório em outro lugar; `origin` é o apelido padrão do principal.
Merge / Rebase	Duas formas de juntar o trabalho de branches diferentes.
Pull Request	Pedido, feito no GitHub, para integrar uma branch em outra, com revisão.

2. Primeiros passos

Conferir instalação e pedir ajuda

```
git --version
git help <comando>      # ex.: git help commit
```

Dizer ao Git quem é você

```
# Para todos os repositórios do computador
git config --global user.name "Aureliano Guimarães"
git config --global user.email "aurelianogbfilho@gmail.com"

# Somente para o repositório atual (sem --global)
git config user.name "Aureliano"

git config --list          # conferir as configurações
```

> A flag `--global` vale para o seu usuário no computador; não tem nada a ver com publicar algo na internet.

Iniciar o versionamento

```
cd caminho/do/projeto
git init                   # cria a pasta oculta .git
git status                 # o comando mais usado: mostra a situação atual
```

Ignorar arquivos com o .gitignore

```
# conteúdo de exemplo do arquivo .gitignore
senhas.txt          # um arquivo específico
*.log               # todos os arquivos .log
node_modules/       # uma pasta
**/temp/            # pastas "temp" em qualquer nível
```

> **Cuidado:** nunca suba senhas, tokens ou chaves. Se o arquivo já foi commitado, colocá-lo no .gitignore não apaga ele do histórico.

3. O ciclo do commit

Preparar (add) e salvar (commit)

```
git add index.html         # prepara um arquivo
git add .                  # prepara tudo da pasta atual
git commit -m "feat: adiciona seção de habilidades"
```

> **Mensagens boas:** curtas, no mesmo tempo verbal e dizendo o *porquê*. No currículo usei prefixos: `feat:` (novidade), `fix:` (correção), `style:` (visual), `docs:` (texto).

Ler o histórico

```
git log                          # completo (q para sair)
git log --oneline                # uma linha por commit
git log -3                       # só os 3 últimos
git log --since="2 days ago"     # por período
git log --after="2026-11-01" --before="2026-11-10"
git log --author="Aureliano"     # por autor
git log --oneline --graph --all  # desenho das branches
git show <hash>                  # detalhes de um commit
```

Comparar versões

```
git diff                  # mudanças ainda NÃO preparadas
git diff --staged         # o que vai entrar no próximo commit
git diff <hash>           # estado atual × um commit
git diff <hash1> <hash2>  # entre dois commits
```

Renomear e remover pelo Git

```
git mv antigo.html novo.html   # já fica preparado para o commit
git rm arquivo.txt
```

Se você renomear ou apagar pelo explorador de arquivos, o Git enxerga "um arquivo apagado + um novo"; aí é preciso rodar `git add -A` antes do commit.

4. Voltando no tempo

Antes de escolher o comando, pergunte: **a mudança já foi preparada? Já foi commitada? Já foi enviada ao GitHub?**

Situação	Comando	Perde trabalho?
Tirar um arquivo da staging	`git restore --staged <arq>`	Não
Descartar edição não preparada	`git restore <arq>` ou `git checkout -- <arq>`	**Sim**
Corrigir a mensagem do último commit	`git commit --amend -m "nova"`	Não
Incluir um arquivo esquecido no último commit	`git add <arq>` + `git commit --amend --no-edit`	Não
Desfazer o último commit mantendo os arquivos	`git reset HEAD~1` (ou `--soft`)	Não
Voltar tudo para o último commit	`git reset --hard HEAD`	**Sim**
Apagar o último commit e suas mudanças	`git reset --hard HEAD~1`	**Sim**
Desfazer um commit já publicado	`git revert <hash>`	Não (cria commit inverso)
Só olhar uma versão antiga	`git checkout <hash>` / `git switch --detach <hash>`	Não

> **Regra de ouro:** commit que já foi para o GitHub se desfaz com `revert`, não com `reset` / `amend` — senão o histórico dos colegas diverge.

5. Branches e Git Flow

Comandos de branch

```
git branch                    # lista (o * marca a atual)
git branch feature/contato    # cria
git switch feature/contato    # entra (antigo: git checkout)
git switch -c feature/contato # cria e entra (antigo: git checkout -b)
git switch main               # volta para a principal
git branch -m main            # renomeia a branch atual
git branch -d feature/contato # apaga (já integrada)
git branch -D feature/contato # força apagar (cuidado!)
```

Juntar trabalho: merge e rebase

```
git switch develop
git merge feature/contato     # traz os commits da feature para develop

git switch feature/contato
git rebase develop            # reaplica a feature por cima de develop
```

Merge preserva o histórico como aconteceu (cria um commit de junção quando precisa). **Rebase** deixa o histórico em linha reta, mas reescreve os hashes — use só em branches que ainda não foram compartilhadas.

Resolvendo conflitos

```
<<<<<<< HEAD
versão da branch atual
=======
versão da outra branch
>>>>>>> feature/contato
```

1) `git status` mostra os arquivos em conflito · 2) edite, escolha o conteúdo certo e apague os marcadores · 3) `git add` · 4) `git commit` (merge) ou `git rebase --continue` (rebase). Desistir: `git merge --abort` / `git rebase --abort`.

O Git Flow que usei (modelo da Asimov Jr)

Branch	Papel	No meu currículo
`main`	Versão estável, publicada.	O que está no ar no GitHub Pages.
`develop`	Onde as novidades se juntam.	Integra as seções antes de publicar.
`feature/*`	Uma funcionalidade nova cada.	`feature/terminal-interativo`, `feature/tema-escuro` …
`release`	Pré-lançamento / revisão final.	Revisão de textos antes da v1.0.0.
`hotfix`	Correção urgente direto da main.	`hotfix/link-linkedin`.

6. Trabalhando com o GitHub

Ligar a pasta local a um repositório

```
git remote -v                                          # já existe vínculo?
git remote add origin https://github.com/USUARIO/REPO.git
git remote set-url origin <nova-url>                   # corrigir URL
git remote remove origin                               # desfazer vínculo (não apaga nada no
GitHub)
```

Enviar e receber

```
git push -u origin main        # 1º envio: publica e "amarra" a branch
git push                       # próximos envios
git push -u origin develop     # publicar outra branch
git push origin --delete <br>  # apagar branch no GitHub

git fetch origin               # baixa novidades SEM misturar
git pull                       # baixa E integra (fetch + merge/rebase)
git pull origin main
git clone <url>                # copia um repositório inteiro para o PC
```

Colaboradores e Pull Request

Convidar alguém: repositório → *Settings* → *Collaborators* → *Add people* → usuário ou e-mail.

Pull Request: depois de `git push -u origin feature/x`, o GitHub mostra o botão *Compare & pull request*. Escolha a base (ex.: `develop`), descreva a mudança, peça revisão e faça o merge. Depois: `git switch develop && git pull`.

7. Extras

Tags (versões de entrega)

```
git tag v1.0.0                              # tag simples
git tag -a v1.0.0 -m "Currículo publicado"  # tag anotada
git tag                                     # listar
git show v1.0.0                             # ver o conteúdo
git push origin --tags                      # tags não sobem sozinhas!
```

Stash (guardar na gaveta)

```
git stash push -m "meio do caminho"   # guarda sem commitar
git stash list
git stash pop                         # traz de volta e remove da gaveta
```

Repositório bare

```
git init --bare central.git
```

Repositório "servidor", sem arquivos de trabalho, que só recebe `push`. No dia a dia o GitHub cumpre esse papel.

GitHub Copilot (benefício de estudante)

Em *education.github.com → Join Global Campus → Student* → escolher UNIFEI, cadastrar e confirmar o e-mail `@unifei.edu.br` e enviar o atestado de matrícula do SIGAA em JPEG/PNG. Aprovado, basta instalar as extensões **GitHub Copilot** e **Copilot Chat** no VS Code e logar.

8. Como publiquei meu currículo

```
# 1. No GitHub: New repository → nome USUARIO.github.io → Public → sem README
cd curriculo-online
git init
git add .
git commit -m "init: estrutura inicial do currículo"
git branch -M main
git remote add origin https://github.com/aurelianogbf/aurelianogbf.github.io.git
git push -u origin main

# 2. Git Flow: develop + uma feature por funcionalidade
git switch -c develop
git push -u origin develop
git switch -c feature/terminal-interativo
#   ...edita, testa...
git add . && git commit -m "feat: terminal interativo"
git switch develop && git merge feature/terminal-interativo
git branch -d feature/terminal-interativo

# 3. Publicar: develop → main + tag de versão
git switch main && git merge develop
git tag -a v1.0.0 -m "Primeira versão do currículo"
git push && git push origin --tags

# 4. No GitHub: Settings → Pages → Branch: main / (root) → Save
```

> Em 1–2 minutos o site fica no ar em `https://aurelianogbf.github.io`. Toda vez que algo novo chega à `main`, o GitHub Pages atualiza sozinho.

9. Cola rápida

Quero...	Comando
Saber o que está acontecendo	`git status`
Preparar e salvar	`git add .` → `git commit -m "msg"`
Ver o histórico resumido	`git log --oneline --graph --all`
Criar e entrar numa branch	`git switch -c <nome>`
Juntar uma branch na atual	`git merge <nome>`
Enviar ao GitHub	`git push` (1ª vez: `git push -u origin <branch>`)
Receber do GitHub	`git pull`

Desfazer commit publicado	`git revert <hash>`
Marcar uma versão	`git tag -a v1.0.0 -m "msg"`

Erros que encontrei pelo caminho

Mensagem	O que significa / o que fazer
`src refspec main does not match any`	Não existe branch `main` local (ou nenhum commit ainda). Faça o primeiro commit e/ou `git branch -M main`.
`remote origin already exists`	O vínculo já existe. Confira com `git remote -v`; corrija com `set-url`.
`rejected … fetch first`	O GitHub tem commits que você não tem. Rode `git pull`, resolva conflitos e faça push de novo. Evite `--force`.
`nothing to commit, working tree clean`	Nada mudou desde o último commit (ou o arquivo está no .gitignore).
`Please tell me who you are`	Falta configurar `user.name` e `user.email`.

> **Fontes:** curso "Git e GitHub Essencial" (Udemy), material da Semana 2 da Asimov Jr, documentação oficial em git-scm.com/docs e docs.github.com/pt.